C语言--流程控制语句04

c语言--高级循环

定义：通过一些语句,控制程序的执行流程

分类，顺序结构 分支结构 循环结构

一.顺序结构

顺序结构:从上往下依次执行,是程序默认的执行过程

二.分支结构

1.if 判断

作用:如果满足条件,才可以执行指定的代码

```
**1.**if(关系表达式)
{
语句体;
}

**2.**if(关系表达式)
{
语句体;
}`else
{
语句体
}`

**3.**if(关系表达式A)
{
语句体A;
}
else if(关系表达式B)
{
语句体B;
}
else if(关系表达式C)
{
语句体C;
}
==else==
{
语句体N;
}
```

PS:

2.switch语句

```
作用:把所有的选择一一列举出来,根据不同的条件任选其一
switch(表达式)

case 值1:

语句体1;
**break;**
case 值2:

语句体2;

break;

{

**default:**

语句体n;
break;

}
```

//值-》比较
PS:

三.循环结构

```
**组成**
初始化语句 条件判断语句 条件控制语句 循环体语句
==种类1·==
==for循环==
for **(int i = 1; i <= 5 ; i[]++)*  * [仅代表次数]

{
printf("Bogeigei\n");
}

PS:
```

==a = a + i;累加计算==
变量的生命周期：只在所属的大括号中有效

如果我想每次循环都操作同一个变量,比如说:累加,变量必须定义在外面
如果我想每次循环都是一个全新的变量,变量可以定义在循环的里面
==种类2·==
==while循环==
初始化语句

while(条件判断语句)
{

循环体语句
条件控制语句

}
（相比for更符合运行逻辑）
for和 while 的区别:(看的清楚)

>for循环中:知道循环次数或者循环的范围

>while循环:不知道循环的次数和范围,**只知道循环的结束条件。**

看知不知道循环结束条件选择
练习1

```c
#include<stdio.h>
int main()
{
    printf("该工具可用于检测一个数是否为2的非负次幂\n请输入一个正整数\n");

    double num1;

    scanf("%lf", &num1);

    int num2 =(int)num1;

    int numa2 = num2;
    //提前定义一个整型变量，使得结果不会成2.000000这样
    double num3 = num1 - num2;
    /*相减取余一定是 double int double
    如果是int int double 会出现5/2=2d的情况*/
    if (num3 == 0&&num2>0)
```

```
        //提前排除小数和负数
    {
        if (num2 == 1)
        {
            printf("%d是2的幂次数\n", numa2);
        }
        else
        {
            double num6 = 0;
            double num7 = 0;
            /*先写一个常量使得第一次可以运行
            再通过改变常量的值num=num/2
            使得之后的循环可以结束*/
            while (num6 == 0 && (num7>1.000001||num7<0.999999))
                //非2次幂最好用区间形式表达
            {
                {
                    num1 = num1 / 2;
                    int num5 = (int)num1;
                    num6 = num1 - num5;
                    num7 = num1;
                    //num7不能是int
                    //这一步使得是1的情况会停止
                    //第一次进入时num1没变，无需定义新变量
                }
            }
            if (num1 == 1)
            {
                printf("%d是2的幂次数\n", numa2);
            }
            else
            {
                printf("%d不是2的幂次数\n", numa2);
            }
        }
    }
    else
    {
        printf("请输入一个正整数\n");
    }
    return 0;
}
```

//最大的总结，int的取整性

种类3：do...while循环

```
初始化语句;

do {

循环体语句;

条件控制语句;

}while(条件判断语句);
（做一次do后再判断循环）**（至少一次）**
```